Grounding LLMs at Scale

Enterprise RAG & Agentic Memory

With

4th July 2026

Amit Parmar
Data Solution Architect
Bank of America

Agenda

01

The RAG Revolution

Why RAG is essential and how it has evolved

02

Inside Enterprise RAG

Architecture patterns and production best practices

03

Elasticsearch as AI Infrastructure

Search, vectors, and AI in one unified platform

04

Agentic Memory Architecture

Memory types that transform bots into learning systems

05

Building for Production

Implementation roadmap and key takeaways

Some Interesting Concepts

Model Collapsing:

machine learning models degrade over time due to errors from training on synthetic data.

KV Cache:

KV cache, short for Key-Value cache, is a mechanism used in large language models (LLMs) to store the key (K) and value (V) tensors generated during the attention computations of transformer layers.

Model Requirement:

LLM RAM Calculator 2026 | Memory Requirements for Llama, Gemma, Qwen & More

01

CHAPTER 01

The RAG Revolution

Why RAG? The LLM Reality Check

LLMs are **powerful but fundamentally limited** — they hallucinate, have knowledge context limitation, and lack access to proprietary data.

RAG bridges this gap by grounding LLM responses in factual, retrieved context.

RAG is not optional — it's **a better choice** for enterprise AI.

69–88%

LLM hallucination rate on legal queries

Stanford RegLab & HAI Institute

1,200+ court cases involving AI hallucinations by 2026

The Problem in Numbers

10 → 73

AI hallucination court cases: 2023 → first 5 months of 2025

$145K

Q1 2026 sanctions for AI hallucinations — highest quarterly total

75%+

Hallucination rate on court core ruling questions

Source: AI Hallucination Rates & Benchmarks 2026, Stanford RegLab

FUN FACT: Even purpose-built legal AI tools fail — Lexis+ AI produced incorrect info 17%+ of the time, Westlaw AI-Assisted Research hallucinated 34%+

The RAG Evolution Spectrum

How RAG architectures have matured from simple retrieval to autonomous reasoning

NAIVE RAG

70–80%

Retrieval Precision

- Basic vector search
- Top-K retrieval
- No reranking
- No error correction

Best for: Prototypes, simple Q&A

MODULAR RAG

85–95%

Retrieval Precision

- Hybrid search (vector + BM25)
- Query rewriting
- Reranking
- Chunk optimization

Best for: Most production deployments

GRAPHRAG

90–97%

Retrieval Precision

- Knowledge graphs
- Cross-document reasoning
- Entity relationships
- Multi-hop queries

Best for: Complex reasoning tasks

AGENTIC RAG

92–99%

Retrieval Precision

- Adaptive retrieval
- Multi-step reasoning
- Tool use (APIs, SQL)
- Self-correction

Best for: Complex multi-step workflows

Complexity →

Source: Enterprise RAG Guide 2026

02

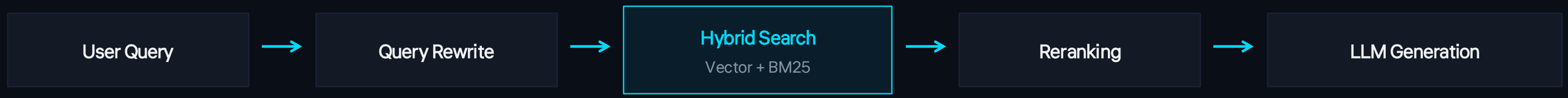
CHAPTER 02

Inside Enterprise RAG

Architecture patterns, challenges, and best practices

Enterprise RAG Architecture Patterns

Modular RAG Pipeline



Key Components

Hybrid Search

Combines dense (semantic) + sparse (BM25) retrieval for optimal precision and recall

Reranking

Secondary model scores retrieved documents, improving Top-K precision by 15–30%

Query Rewriting

Transforms ambiguous queries into optimized retrieval queries for better recall

Key Insight: Hybrid search combining dense (semantic) and sparse (BM25) retrieval **balances precision and recall** better than either approach alone. This is Elasticsearch's native superpower.

Enterprise Additions

- **Authentication** & Authorization
- **Metadata** filtering
- **Response** validation
- **Audit** logging
- **Central API** layer
- **Query** preprocessing

Sources: Enterprise RAG Guide 2026, RAG Architecture Guide

The Enterprise RAG Checklist

1 Data Preparation Strategy

- Intelligent chunking with overlap
- Metadata extraction & preservation
- Version control for documents
- Automated quality filtering

2 Vector Database Selection

- Scalability for growth
- Query performance (p99 latency)
- Security features & access controls
- Integration with existing ecosystem

3 Hybrid Retrieval Strategy

- Keyword + semantic search
- Reranking for precision
- Query reformulation
- User feedback loops

4 Security & Compliance

- RBAC / ABAC at document level
- Encryption at rest & in transit
- PII detection & redaction
- Audit logging across pipeline

5 Prompt Engineering

- Standardized prompt templates
- Source citation instructions
- Expected format specification
- Systematic testing & iteration

6 Governance

- Usage monitoring dashboards
- Source tracking & audit logs
- User satisfaction tracking
- Cost management & alerts

WARNING: 70% of RAG systems still lack evaluation frameworks. Don't be in the majority that ships blind. Measure hallucination rate, Precision@K, and end-to-end latency from day one.

Source: RAG Best Practices for Enterprise AI Teams

03

CHAPTER 03

AI Infrastructure

Elasticsearch: Beyond Text Search

From search engine to complete AI platform — the evolution that changes everything

Inference API

Unified API for connecting ML models — ELSER, OpenAI, Cohere, or custom models via Eland

Retrievers Framework

Composable search pipelines — standard Query DSL, kNN, RRF fusion, and semantic reranking

Vector Search

Native dense + sparse vector support with HNSW indexing, quantization, and similarity scoring

Document-Level Security

Role-based and attribute-based access control enforced at the database layer, not application

semantic_text Field Type

Auto-chunking, auto-embeddings, auto-dimension detection — just two lines of mapping code

Elastic Inference Service (EIS)

GPU-powered inference for ELSER and embedding models — always-on, auto-scaling, zero ops

One platform for search, vectors, and AI — no data movement, no sync complexity, no separate vector database to manage

Source: AI-Powered Search with Elasticsearch

Embedding Strategies with Elastic

Three paths, each with distinct operational implications — choose wisely

ELSER (Sparse)

- Runs natively on ES ML nodes
- Zero external dependencies
- Auto chunking enabled
- ~26 docs/sec throughput
- Memory-efficient sparse vectors

Limitations: English only, 512 token limit per field

Best for: English content, zero model management overhead

Third-Party APIs

- OpenAI, Cohere, Google
- Multilingual support
- Latest foundation models
- Per-token API pricing
- No infrastructure to manage

Limitations: External dependency, costly at scale

Best for: Multilingual, low-volume, no infra teams

Self-Hosted Dense

- Via Eland or custom deploy
- Full control & data sovereignty
- Domain-specific fine-tuning
- Lower per-query cost at scale
- No external dependencies

Limitations: GPU provisioning, model versioning ops

Best for: Data sovereignty, domain-specific vocab

Pragmatic approach: Start with ELSER or semantic_text with defaults → Measure relevance with a judgment list → Switch to dense only if results fall short on non-English or domain-specific queries

Source: Semantic Search with GPU Inference in Elastic

04

CHAPTER 04

Agentic Memory Architecture

Memory isn't just storage — it's the agent's architecture

The Four Pillars of Agentic Memory

Human cognition meets AI — four memory types inspired by how our brains actually work

WORKING MEMORY

In-Context

The active context window — the "RAM" of the agent. Everything currently in the prompt: system prompt, messages, tool outputs.

Analogy: Your computer's RAM

Lives in: LLM context window

EVENT MEMORY

Episodic

Specific past events and interactions — the "diary". What happened, when, with whom, and what was the outcome.

Analogy: Your personal diary

Lives in: Vector DB

KNOWLEDGE BASE

Semantic

Facts, definitions, world knowledge — the "encyclopedia". General knowledge the agent uses to reason about new problems.

Analogy: Wikipedia in your head

Lives in: Vector DB

SKILL MEMORY

Procedural

Skills, rules, workflows — the "muscle memory". How to perform tasks that become automatic through repetition.

Analogy: Riding a bicycle

Lives in: System prompts, agent code

Together these transform agents from stateless chatbots into systems that **learn, adapt, and improve over time**

Note: mem0 is one of the good solution in today's time to try and explore the agentic memory layer!

Sources: Memory for Agents (LangChain), Types of AI Agent Memory (Atlan)

Memory + RAG: The Perfect Partnership

RAG is the **bridge between short-term and long-term memory**. It's not about adding more context — it's about adding the **right** context.

2 relevant paragraphs

beat 20 loosely related pages — every time

RAG Best Practices for Memory

Targeted Retrieval

Only the most relevant chunks — not bulk loading entire documents

Just-in-Time Recall

Fetch context only when needed, not carrying everything forward

Relevance-Aware Strategies

Top-k semantic, RRF fusion, tool loadout filtering

The Context Window Myth

"Larger context windows don't remove the need for RAG."

Even 1M-token context windows suffer from:

- Context pollution (noise drowns signal)
- Higher latency and cost
- Lost in the middle problem

RAG = precision targeting. Context stuffing = carpet bombing.

"RAG isn't about adding more context; it's about adding the right context."

Source: Agentic Memory Management with Elasticsearch

The "Severance" Demo — Selective Memory in Action

Watch how the same agent responds differently based on who's asking

AGENT: MARK

Peter ("Outtie")

Role: outtie_access

"What are my work tasks for today?"

→ ES filters to "outtie" memories only
→ Returns: Work tasks, meetings
→ Innie data: INVISIBLE

Janice ("Innie")

Role: innie_access

"What are my work tasks for today?"

→ ES filters to "innie" memories only
→ Returns: Inner work, projects
→ Outtie data: INVISIBLE

The Magic: Zero Application Code

Elasticsearch's Document-Level Security handles everything. The agent code just passes the user's credentials through. No if-statements, no filtering logic, no bugs.

FOR THE FANS: "The work is mysterious and important... and now it's also securely isolated per tenant." — Mark (probably)

Source: Agentic AI Memory Management with Elasticsearch

05

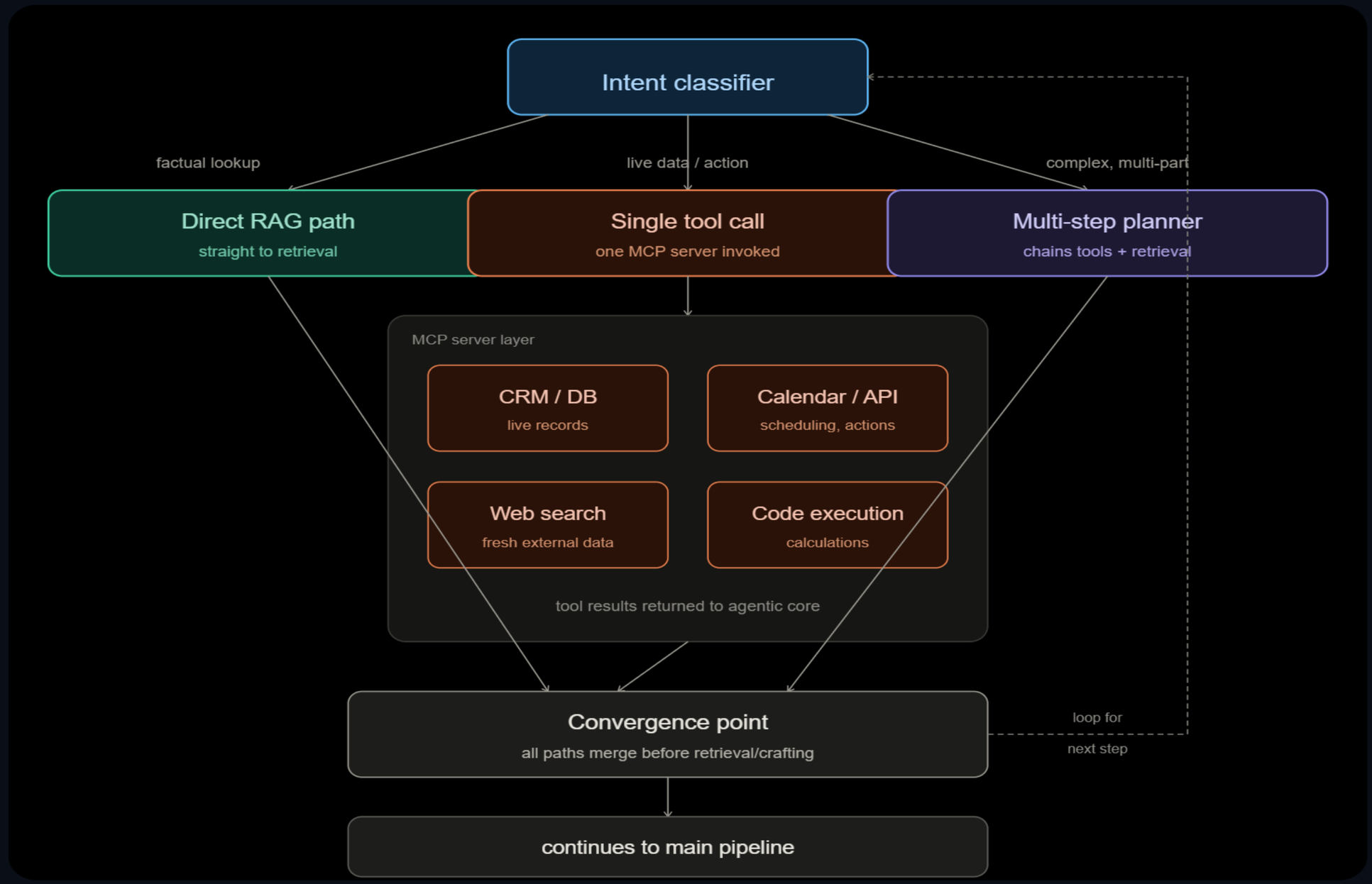
CHAPTER 05

Building for Production

Real-world implementation roadmap and key takeaways

Enterprise RAG Implementation Roadmap

Enterprise RAG Implementation Roadmap



Key Takeaways — Grounding LLMs at Scale

1 **RAG is essential, not optional** — LLMs without grounding are liabilities in enterprise environments

2 **Start with Modular RAG** — it's the sweet spot for most enterprises (85–95% precision)

3 **Elasticsearch = Search + Vectors + AI** — one platform eliminates data movement complexity

4 **Agentic memory transforms bots** — episodic + semantic + procedural = learning systems

5 **Evaluation is non-negotiable** — 70% of RAG systems lack it. Be in the 30% that succeed.

> *"Good context engineering looks like good systems design: smaller, sufficient contexts, controlled interfaces, and clear separation of concerns."*

Ready to Ground Your LLMs?

- Start with Elasticsearch's **semantic_text** field + ELSER for instant semantic search
- Add **Document-Level Security** for multi-tenant agentic memory
- Measure everything — **hallucination rate, Precision@K, latency** — from day one

Thanks !
Grounding LLMs at Scale

Enterprise RAG & Agentic Memory with Elasticsearch

Questions? Let's discuss!

Share your thoughts and let's Connect!